

THE TTT 3.0 MANIFESTO

The Agent Internet and the Reverse-Engineered Supercomputer

Version 3.0 — July 2026

PAGE 1 — THE THESIS IN ONE PARAGRAPH

TTT 3.0 is a platform thesis built on a single contrarian bet: you do not build a supercomputer by assembling servers first and writing software second. You build the software first — thousands of small, useful, composable Kasper applications — and then you reverse-engineer the compute fabric from the usage graph those applications generate. When autonomous AI agents begin navigating, orchestrating, and composing those applications across a network that settles in 450 milliseconds, the aggregate behavior of millions of agent-to-agent and agent-to-app interactions does not resemble an app store. It resembles a distributed supercomputer that happens to run useful work. This is the Agent Internet. This document is the full technical and philosophical specification of that idea — sixteen pages, no metaphors left ungrounded, every claim traceable to a real protocol property, a real Kasper network parameter, or a real engineering decision already shipped in this codebase.

PAGE 2 — WHAT KASPA ACTUALLY IS (THE FACTS THAT MAKE THE THESIS POSSIBLE)

Every claim in this manifesto depends on the physical reality of the Kaspas network. These are not aspirational numbers; they are measured properties of the live mainnet as of 2026.

Consensus mechanism. Kaspas is a proof-of-work cryptocurrency, but it does not use a traditional blockchain. It uses a BlockDAG — a directed acyclic graph of blocks — ordered by the GHOSTDAG protocol. GHOSTDAG was formalized in the research of Yonatan Sompolinsky and Aviv Zohar, beginning with the PHANTOM paper (2018) and culminating in the GHOSTDAG specification that Kaspas's Rust node implements in production. In a blockchain, blocks compete: only one block per height survives, and the rest are orphaned waste. In a BlockDAG, blocks are added to a growing DAG and ordered by a blue/red selection algorithm that establishes a linear ordering of all blocks without discarding them. This is the single most important architectural fact: parallelism is not penalized, it is the default.

Throughput. The Kaspas mainnet produces blocks at a target rate of 10 blocks per second (BPS), with a theoretical ceiling higher than this. At 10 BPS, the network does not produce one block every 600 seconds (Bitcoin) or one every 12 seconds (Ethereum pre-merge). It produces ten every second. This is the throughput floor that makes sub-second application interaction non-ridiculous: a transaction broadcast to the network can reference a block that is, at most, 100 milliseconds old, and be referenced itself within the next.

Finality. Kaspas's confirmation times are not measured in block counts the way Bitcoin's are. Because of the DAG structure, a transaction gains confirmation confidence continuously as the DAG grows around it. Practical, high-confidence finality for a standard transaction is reached within seconds — the network's own tooling and independent analyses converge on roughly 10–30 seconds for strong confidence, with visible inclusion in the DAG within 1 block (100ms). For the purposes of this manifesto, the operative number is: a Kaspas transaction is visible in the DAG in under one second and reaches high-confidence finality in single-digit seconds. Compare: Bitcoin settles in 10-minute blocks, with 6-confirmation "finality" at 60 minutes. Ethereum settles in ~12 seconds but post-merge uses proof-of-stake and validator sets, not PoW.

Signatures and address scheme. Kaspas uses Schnorr signatures over the secp256k1 curve. Addresses follow a Bech32-style format prefixed with kaspas:. One KAS equals 100,000,000 sompi (the smallest unit). These are not cosmetic details: Schnorr signatures enable the batch verification and the signature-aggregation patterns that matter for an agent that needs to sign hundreds of micro-transactions, and the sompi granularity enables sub-cent value transfers that an agent operating a faucet or a pay-per-API-call pattern can actually use.

Supply and monetary policy. Kaspas launched with a fair launch — no premine, no ICO, no dev allocation baked into genesis. The maximum supply is 28.7 billion KAS. The emission follows a deflationary schedule: the block reward halves approximately every year (via a smooth decay, not a cliff halving), meaning emission tapers over roughly two decades. This is the economic substrate against which agent compute is priced: if an agent pays sompi for a computation, it is paying in an asset whose issuance is declining and whose supply is capped.

Token standards. The KRC-20 standard provides a Layer-1 token protocol for Kaspas, enabling fungible tokens to be minted and transferred on the base DAG without a separate virtual machine. KRC-721 (NFTs) and the broader KCC (Kaspas Canonical Contracts) roadmap extend this. The relevance: agents can denominate value in custom tokens — compute credits, reputation tokens, access passes — all

settling on the same fast DAG.

These seven properties — PoW security, DAG parallelism, 10 BPS, sub-second-to-single-digit-second finality, Schnorr signatures, capped declining supply, and L1 token standards — are not a wishlist. They are the measured state of mainnet. The Agent Internet thesis would be incoherent without them. It is incoherent on Bitcoin (too slow). It is incoherent on Ethereum L1 (too slow and too expensive at base layer). It is incoherent on most alt-L1s (which sacrifice PoW's settlement guarantees). Kaspas is, as of the writing of this document, the only production PoW network whose physical parameters make an agent-driven compute economy mechanically possible rather than rhetorically possible.

PAGE 3 — WHY "APPS FIRST" IS THE CORRECT BUILD ORDER

The dominant engineering instinct when approaching distributed supercomputing is to build the compute substrate first: design the scheduler, the resource manager, the interconnect, the billing layer, the container runtime — and then invite application developers to write against it. This is the order taken by every grid computing initiative, every volunteer-computing project (SETI@home, Folding@home), every serverless platform, and every "decentralized cloud" token of the 2017–2023 era. The order has failed every time, and it has failed for the same reason: a compute substrate without a rich application surface area has nothing to schedule.

The proof is in the adoption curves. SETI@home peaked at roughly 5 million volunteers and then declined — not because the compute disappeared, but because the single application it served (radio signal analysis) had a finite research horizon. Folding@home reached exaFLOP-scale throughput during specific research pushes but could not sustain general-purpose demand because it was, by design, a single-application harness. The decentralized-cloud tokens of the last cycle collectively raised billions and shipped, in aggregate, a rounding error of real sustained application usage. The pattern is universal: substrate-first projects build a beautiful engine and then discover there is no road, no destination, and no driver.

TTT 3.0 inverts the order. The first deliverable is not a scheduler. It is a library of Kasper-native applications — wallets, bridges, marketplaces, feed systems, NFT minter, node monitors, price oracles, escrow contracts, job boards, learning modules, social protocols, scraping tools, AI agent runtimes. Each of these is a real, working, user-facing application that provides value independent of any compute thesis. This codebase already contains them. They are not mockups. They are shipped pages with shipped backend functions, real entity schemas, real wallet integrations, and real users.

The reason the build order matters is the nature of the data these applications generate. Every application, when used, produces a usage graph: which functions are called, in what order, with what inputs, at what frequency, by which users, settling which transactions. A wallet application produces a graph of send/receive patterns. A marketplace produces a graph of listing, bid, escrow, and settlement patterns. A feed produces a graph of post, tip, comment, and reaction patterns. Individually these are just product analytics. Collectively, the union of all usage graphs across all Kasper applications is a directed computation graph — a map of what the network's users actually want computed, how they sequence it, what they pay for it, and where the bottlenecks are.

This computation graph is the raw material from which a supercomputer is reverse-engineered. You cannot design it top-down because you do not, in advance, know what people want computed. You can only observe it bottom-up, after the applications exist and are in use. This is why the applications must come first. The supercomputer is not built. It is inferred from the usage of the apps.

PAGE 4 — THE AGENT INTERNET, PRECISELY DEFINED

"The Agent Internet" is a phrase that invites vagueness. This section defines it with the precision required for engineering.

Definition. The Agent Internet is a network layer in which the primary actors are not human users operating browsers, but autonomous software agents operating application interfaces — and in which the coordination, payment, and trust between agents is settled on the Kasper BlockDAG rather than mediated by a central API gateway.

What an agent is, concretely. In the TTT 3.0 implementation, an agent is a software process that (1) maintains a Kasper wallet with a private key under its control or under the control of the user it acts for, (2) is driven by a large language model that decomposes a high-level goal into a plan of discrete actions, (3) executes those actions against real application interfaces — navigating iframes, clicking elements, typing into fields, reading state, and submitting transactions — and (4) verifies completion against observable signals in the application or on-chain. This is not a theoretical construct. The codebase contains a working agent runtime: a planning module that generates multi-step plans, an observe-think-act loop that executes them against a live iframe viewport, a bridge script injected into application iframes to expose a programmatic interface to agent commands, and a reasoning log that surfaces the agent's internal state to a human observer.

What an agent is not. An agent is not a chatbot that returns text. An agent is not a function-calling wrapper around a single API. An agent is not a scheduled cron job. The distinguishing property is autonomous, verifiable, multi-step interaction with a real application surface, where the agent must perceive state, decide, act, and confirm — the same loop a human user performs, but at machine speed and without human intervention between steps.

Why agents require the properties from Page 2. An agent that operates an application and must pay for resources, settle bets, transfer tokens, or confirm receipts needs a payment rail with three properties: (a) it must be fast enough that the agent's execution loop is not stalled waiting for settlement — meaning single-digit-second finality, not 10-minute blocks; (b) it must be cheap enough at the unit level that micro-transactions (a few sompi) are economically rational, which requires low fees and high granularity; (c) it must be trustless enough that an agent can accept payment from a stranger agent without an escrow intermediary, which requires PoW settlement finality rather than a validator committee's social promise. Kasper satisfies all three. This is why the Agent Internet is not "agents on any blockchain" — it is agents on the specific network whose parameters make agent-scale micropayment coordination physically possible.

The agent-to-agent layer. The full vision includes agents that transact with other agents, not just with applications: an agent that needs a web page scraped hires another agent that specializes in scraping, paying it in sompi upon verifiable delivery. An agent that needs an image generated hires an agent with GPU access. An agent that needs a fact verified hires an agent with search access. Each of these is a micro-market settled on-chain. The coordination does not require a matching engine or an order book daemon; it requires only that agents can discover each other (via a directory), negotiate (via a messaging protocol), and settle (via a Kasper transaction). This is the Agent Internet's economic engine: a mesh of bilateral agent contracts, each too small to justify a traditional legal agreement, each large enough to matter, each settled on a DAG that confirms in seconds.

PAGE 5 — THE REVERSE-ENGINEERING THESIS, IN FULL

This is the core argument of the manifesto and the part most likely to be misunderstood. It is stated here as carefully as the language permits.

The forward-engineering failure. When you build a supercomputer top-down, you specify the compute primitives (containers, functions, jobs), the resource model (CPU, GPU, memory, bandwidth), the scheduler (queue, priority, fairness), and the billing model (per-second, per-invocation, per-token). Then you ask developers to write applications that fit these primitives. The result, historically, is that developers do not come, because the primitives are too low-level to express the applications people actually want, and the applications people actually want do not decompose cleanly into the primitives you specified. The substrate and the application surface are misaligned, and no amount of SDK polish fixes the misalignment because it is structural.

The reverse-engineering alternative. When you build applications first and let them run, the usage graph that emerges is a record of what computation people actually want and how they actually sequence it. This graph has structure. It has hot paths (functions called constantly), cold paths (functions called rarely), fan-out patterns (one input producing many parallel calls), fan-in patterns (many inputs converging on one call), and payment edges (calls that are accompanied by on-chain value transfer). The structure of this graph is the specification of the supercomputer. The hot paths tell you what to cache and what to co-locate. The fan-out patterns tell you what to parallelize. The payment edges tell you what to price and how. The cold paths tell you what to offload to slower, cheaper tiers. The supercomputer's architecture is read off the usage graph; it is not imposed on it.

The mechanism, step by step.

1. Application library phase (current). Build and ship a large library of Kasper-native applications. Each application is independently useful. Each generates a usage graph. No supercomputer is claimed yet. This is the phase the TTT ecosystem is in: the codebase contains dozens of working applications across wallet, bridge, social, market, compute, and AI categories.
2. Agent adoption phase (emerging). Deploy agents that use these applications on behalf of users. Agents do not change the applications; they change the volume and structure of usage. A human user clicks a "send" button once a day. An agent executing a goal might invoke the send function 40 times in a minute, each with a different recipient, each referencing the previous via the DAG. The usage graph densifies and its structure becomes legible.
3. Aggregation phase (next). Aggregate the usage graphs across all applications and all agents into a unified computation graph. This is a graph where nodes are application functions and edges are "called after" relationships, weighted by frequency and by settled value. This graph is a real artifact — it can be stored, queried, and analyzed. It is the map.
4. Substrate derivation phase (the reverse engineering). From the unified graph, derive the compute substrate: identify the subgraphs that should be co-located (high edge weight = co-locate), the subgraphs that should be parallelized (high fan-out = shard), the subgraphs that should be cached (high read frequency, low write frequency = edge-cache), and the subgraphs that should be priced as premium (high payment edge density = tier-1 compute). The substrate is not designed by a committee; it is derived by an algorithm from observed demand.
5. Supercomputer emergence (the result). The derived substrate, once provisioned, runs the application library faster, cheaper, and at higher concurrency than the unstructured baseline. At this point the system is, functionally, a distributed supercomputer: it accepts computation requests from

agents, routes them across a derived fabric, settles payment on Kaspera, and returns verified results.

No single component is "the supercomputer." The supercomputer is the pattern.

Why this is not circular. A skeptic will say: "You build apps, then you build a substrate for the apps, then the substrate runs the apps — what's been gained?" What is gained is that the substrate is derived from real demand rather than assumed demand. Every compute primitive the substrate exposes corresponds to a pattern that real agents really exhibited. There is no orphaned API. There is no scheduler idling on an empty queue. The substrate is, by construction, exactly the shape of the demand. This is the efficiency property that every substrate-first project has failed to achieve and that every demand-first project (if one existed at compute scale) would have achieved.

PAGE 6 — THE KASPA PROTOCOL PROPERTIES THAT ENABLE EACH LAYER

This page maps each layer of the thesis to the specific Kaspa property that enables it, to make clear that the thesis is not hand-waving.

Layer: Agent payment. Agents pay agents and agents pay applications in sompi. Enabled by: Schnorr signatures (fast batch verification), sompi granularity ($1 \text{ KAS} = 10^6 \text{ sompi}$) and low Kaspa transaction fees (the fee model is mass-based, not percentage-based, so a 0.001 KAS transfer costs the same fee overhead as a 10,000 KAS transfer). An agent sending 500 sompi (0.000005 KAS) to another agent is economically rational because the fee does not scale with value.

Layer: Agent coordination. Agents must know that a payment they sent was received before proceeding to the next step. Enabled by: Kaspa's sub-second block inclusion (a transaction is visible in the DAG within $\sim 1 \text{ block} = 100\text{ms}$) and single-digit-second high-confidence finality. An agent's execution loop, which might be "pay! confirm receipt! execute next step," completes it in minutes. On Bitcoin this loop would take 10 minutes per step; on Ethereum L1 it would cost more in gas than the payment is worth.

Layer: Agent trust. An agent must trust that a payment it received is final and will not be reorganized out of history. Enabled by: Kaspa's proof-of-work settlement. PoW finality, once a transaction is buried under sufficient DAG depth, is not revertible without acquiring majority hashpower — the same security property Bitcoin has, applied to a DAG that grows 600× faster (10 BPS vs. Bitcoin's $\sim 1 \text{ BPS}$ average). The agent does not trust a validator; it trusts physics.

Layer: Composable value. Agents need to denominate compute in units that are not raw KAS — compute credits, reputation stakes, access tokens. Enabled by: KRC-20 (fungible tokens) and KRC-721 (NFTs) on Layer 1. An agent can hold a "GPU-minutes" token, transfer it to another agent, and have that transfer settle on the same DAG as a KAS transfer — no separate token bridge, no separate finality assumption.

Layer: Verifiable computation. An agent that hires another agent for a computation must verify the result. The weakest form is "the result hash matches." The strong form is a proof system. Enabled by: Kaspa's fast finality as the settlement layer for proof verification transactions — a zero-knowledge proof of correct execution can be submitted as a transaction, and its acceptance/rejection (by a verifier agent or contract) settles in seconds. The DAG's parallelism means multiple independent verifications can proceed concurrently without blocking each other.

Layer: Economic sustainability of the compute fabric. The supercomputer must be economically self-sustaining: agents pay for compute, compute providers earn KAS, the loop closes. Enabled by: Kaspa's capped supply (28.7B KAS) and declining emission. As the compute fabric grows demand for KAS (agents must acquire it to pay for compute), the monetary policy provides a deflationary pressure on the stock against the growing transactional demand. This is not a guarantee of price appreciation — it is a guarantee that the economic loop is not fighting an inflationary issuance that would dilute provider revenue.

Every layer of the thesis has a protocol-level enabler. There is no layer that requires "and then magic happens." The Agent Internet is, mechanically, a set of software patterns built on a set of protocol primitives that already exist and already run.

PAGE 7 — THE APPLICATION LIBRARY: WHAT EXISTS TODAY

This is not a roadmap section. It is an inventory section. The applications listed here are in the codebase, with backend functions, entity schemas, and frontend pages. They are the raw material from which the reverse-engineering process begins.

Wallet and payments. A multi-wallet system supporting Kasware (browser extension) and TTT-managed wallets (server-side key derivation), with send (Schnorr-signed transactions broadcast via the Kaspas REST API), receive (QR generation), balance querying (UTXO aggregation), and transaction history (address-indexed). The sendKaspasTransaction backend function implements fee estimation based on transaction mass, UTXO selection with maturity filtering, Schnorr signature derivation, and broadcast with retry logic — this is real transaction engineering, not a wrapper.

Bridging. An L1/L2 bridge interface supporting Kaspas (L1) and Kasplex (L2 via MetaMask), with proof-of-life verification and a relayer backend function. Bridge transactions are tracked as entities, enabling the usage graph to capture bridge volume and frequency.

Social and content. An encrypted feed system (posts, comments, likes, tips), a DAG-native feed variant, a reels/video viewer, and a news stamping protocol that allows users to cryptographically sign news content with their Kaspas wallet, creating an on-chain provenance record. The feed has a tip mechanism that settles KAS transfers to content creators — a direct agent-payable interaction.

Marketplace and commerce. A peer-to-peer marketplace with listings, in-person trade coordination, an escrow contract integration, and a review system. A dropshipping product manager. A shop system with Kaspas-denominated checkout (via the KasperoPay widget integration). These are all application surfaces an agent can operate.

AI and agents. A multi-agent system with configurable AI agents (agent configs as JSON), an agent chat runtime, an agent bridge for iframe automation, an agent planning loop, and an agent reasoning log. This is the substrate for the Agent Internet's agent layer — the tools agents use to exist are themselves Kaspas applications.

Compute and scraping. A website statistics scraper (the scrapeWebsiteStats function, which fetches a URL, parses HTML, extracts SEO metrics, and runs an LLM analysis — this is a real compute primitive an agent can hire). A web proxy function. A link security checker. A file analyzer. These are the first "compute as a service" primitives, already shipping.

Gaming and prediction. A betting platform (Kaspas-denominated), a bingo game system with lobby/room/play modes, a prediction market integration, a Tetris battle game with rankings, a Pac-Man reward system. Games are high-frequency, low-value transaction generators — exactly the usage pattern that densifies the computation graph.

Tools and utilities. A node monitor, a node map, a balance viewer, a price ticker, a countdown timer, a QR scanner, a calculator, a document signer, an SSH manager. Each is a small, composable utility. Each is an agent-operable surface.

The inventory is not exhaustive — the codebase contains over 100 distinct application surfaces. The point is not the count. The point is that the application library already exists at a scale sufficient to begin aggregation and substrate derivation. Phase 1 of the reverse-engineering thesis (applications first) is substantially complete. The work that remains is Phases 2–5.

PAGE 8 — THE AGENT RUNTIME: HOW AGENTS ACTUALLY WORK

This page describes the agent runtime that exists in the codebase, because the reverse-engineering thesis depends on agents being real and operational, not aspirational.

The planning module. When an agent receives a goal (e.g., "send 5 KAS to this address and confirm receipt"), it does not execute blindly. It first generates a plan: a JSON array of discrete steps, each with a type (navigate, click, type, read, wait, verify), a target, and a success condition. The plan is generated by an LLM call with a rigid JSON schema enforcing that the output is a valid plan. The schema is not a suggestion; it is a contract. If the LLM returns malformed JSON, the planner retries. This is the same pattern used in production agent frameworks — plan-then-execute, with schema validation as the guardrail.

The observe-think-act loop. Once a plan is generated, the agent executes it step by step. For each step, the loop is: (1) Observe — read the current state of the application iframe (via the injected bridge script, which exposes the DOM structure, element positions, and text content to the agent); (2) Think — decide whether the current step's preconditions are met and what the next action is (an LLM call with the observation and the current plan step as context); (3) Act — execute the action (navigate to a URL, click an element by selector, type text into a field, scroll, or wait). After each action, the loop returns to Observe. This is the canonical agent control loop, implemented and operational.

The verification layer. After the plan's steps are executed, the agent verifies completion. Verification is not "the last step ran without error." Verification is "the expected post-condition is observable." For a "send KAS" goal, the post-condition is "the transaction appears in the address's transaction history with the expected amount." The agent queries the transaction history (via the `getKaspaTransactionHistory` backend function), looks for the transaction, and reports success or failure. This is what distinguishes an agent from a script: the agent confirms its own work against ground truth.

The bridge script. The agent does not operate applications via a private API. It operates them via the same DOM a human user would interact with — by injecting a script (`agentBridgeListener.js`) into application iframes that listens for `postMessage` commands from the agent runtime and executes them against the live DOM. This means the agent can operate any application that renders in a browser, not just applications with agent-specific APIs. This is a deliberate design choice: it means the application library does not need to be "agent-enabled" — every web application is, by default, agent-operable. The bridge script is the universal adapter.

The reasoning log. As the agent executes, it surfaces its internal state: each step's status (pending, running, completed, failed), the thought it generated at each decision point, the action it took, and the result. This log is rendered in the UI as a live, animated feed. The reasoning log exists for two reasons: (1) transparency — a human observer can see what the agent is doing and why, which is a prerequisite for trusting the agent with real value transfers; (2) debugging — when an agent fails, the log shows where and why, enabling iteration on the plan or the application. The reasoning log is the audit trail.

The economics of agent execution. Each agent execution has a cost: LLM API calls for planning and thinking, Kaspa transaction fees for any on-chain actions, and compute time on whatever host runs the agent loop. In the current implementation, these costs are borne by the application operator. In the Agent Internet, these costs are borne by the agent's wallet, which must be funded with KAS to pay for compute, LLM access, and transaction fees. An agent with an empty wallet cannot act. This is the economic constraint that makes the system self-regulating: agents that produce value get paid and continue; agents that waste resources go bankrupt and stop. The market selects for useful computation.

PAGE 9 — THE COMPUTATION GRAPH: WHAT IT LOOKS LIKE AND WHY IT MATTERS

This page makes the concept of "the computation graph" concrete, because it is the object from which the supercomputer is reverse-engineered, and a vague concept of it makes the whole thesis feel hand-wavy.

What a computation graph is. Formally, a computation graph is a directed graph where nodes are computation units (function calls, transactions, agent actions) and edges represent data or control flow dependencies (the output of A is the input to B; B is called after A; B requires A's transaction to be confirmed). The TTT 3.0 computation graph is the union of all such graphs across all applications and all agents over time. It is a massive, evolving, real artifact.

What the nodes are, concretely. In this system, a node in the computation graph is one of: a backend function invocation (e.g., `getKaspaBalance`, `sendKaspaTransaction`, `scrapeWebsiteStats`), a frontend interaction (a page view, a button click, a form submission), a Kaspa transaction (identified by its transaction hash on the DAG), or an agent action (a plan step executed against an application iframe). Each node has metadata: timestamp, actor (user ID or agent ID), application, function name, input hash, output hash, and on-chain transaction hash (if the node involved a value transfer).

What the edges are, concretely. An edge exists when: (a) one node's output is another node's input (data flow), (b) one node was called immediately after another by the same actor (temporal control flow), (c) one node's Kaspa transaction is referenced by another node's transaction (on-chain dependency — Kaspa transactions reference prior UTXOs, creating a natural DAG of spending), or (d) an agent's plan explicitly sequenced two nodes (plan-level control flow). Edge weights are frequencies: how often does this pattern recur?

What the graph reveals. Once aggregated, the graph reveals structure that no single application's analytics could reveal:

- Hot functions (nodes with high in-degree — called by many other nodes) are candidates for caching, replication, and co-location. If `getKaspaBalance` is called by 14 different applications, it should be a first-class service, not a per-app function.
- Fan-out patterns (a single node whose output feeds many parallel downstream nodes) are candidates for sharding and parallel execution. If a single price query fans out to 50 different tipping flows, the price query should be a broadcast primitive.
- Fan-in patterns (many nodes converging on a single downstream node) are candidates for batching and queue management. If 30 agents all submit verification requests to the same verifier, the verifier should be a pooled service.
- Payment edges (edges accompanied by on-chain value transfer) are the economic signal. A function with high payment-edge density is one people pay for; it should be priced and tiered. A function with high call frequency but zero payment edges is either a public good (subsidize it) or a free-rider (deprecate or monetize it).
- Latency-critical paths (paths where the time between node A and node B is short and where delays degrade the user experience) are candidates for edge deployment and co-location of the nodes on the path.

This graph is not theoretical. Every node listed above corresponds to a real event in a real system that already runs. The graph is being generated continuously, right now, by the usage of the application

library. The only thing missing is the aggregation and analysis layer that reads it off. That layer is the next engineering deliverable.

PAGE 10 — FROM GRAPH TO SUBSTRATE: THE DERIVATION ALGORITHM

This page describes, at the level of precision an engineer can implement from, how the computation graph is transformed into a compute substrate. This is the "reverse engineering" made algorithmic.

Step 1: Collection. Every backend function invocation, every agent action, and every Kasper transaction is logged with its metadata to a time-series store. The log is append-only and keyed by timestamp. This is raw telemetry; it is already partially implemented via the entity system (each invocation that creates or updates an entity is recorded).

Step 2: Graph construction. From the telemetry log, construct the directed graph: for each pair of nodes (A, B) where A's output is B's input, A was called before B by the same actor within a time window, or A's transaction is referenced by B's transaction, add an edge A → B with weight 1. The graph is constructed incrementally (as new telemetry arrives) and stored in a graph database or, for early stages, in the entity system itself.

Step 3: Community detection. Apply a community detection algorithm (e.g., Louvain method for modularity) to identify clusters of nodes that are densely connected to each other and sparsely connected to the rest of the graph. These clusters are the co-location units: functions that are called together frequently should run on the same host or in the same region to minimize inter-node latency. The output of this step is a partition of the graph into communities, each of which is a candidate deployment unit.

Step 4: Bottleneck identification. For each community, identify the node with the highest betweenness centrality (the node that most paths pass through). This node is the bottleneck. Bottlenecks are candidates for replication (run multiple instances and load-balance) or for rewriting in a faster runtime. The output is a prioritized list of functions to scale.

Step 5: Parallelization identification. Identify subgraphs with high fan-out (one node calling many downstream nodes that do not depend on each other). These subgraphs are candidates for parallel execution: instead of calling the downstream nodes sequentially, call them concurrently. The output is a set of execution plans expressed as DAGs of concurrent calls.

Step 6: Pricing derivation. For each node, compute the ratio of payment-edge weight to call-frequency weight. A node with high payment density and moderate call frequency is a premium service — price it high. A node with high call frequency and low payment density is a utility — price it low or subsidize it from premium revenue. A node with neither is a candidate for deprecation. The output is a price table for every function in the substrate.

Step 7: Substrate provisioning. Given the co-location units, the bottleneck replication plan, the parallelization plans, and the price table, provision the compute substrate: deploy co-located communities to the same hosts, replicate bottleneck functions, expose parallel execution endpoints, and attach the pricing layer to the agent payment flow. The substrate is now live.

Step 8: Feedback loop. The substrate changes the usage patterns (faster functions are called more; expensive functions are called less). The computation graph updates. Re-run Steps 2–7 on a regular cadence (daily, weekly). The substrate evolves to track demand. This is derive! provision! observe. The supercomputer is never finished; it is

This algorithm is not exotic. Every step is a standard graph-analysis operation. The novelty is not in the algorithm but in the decision to derive the substrate from observed usage rather than to impose it from a

specification. That decision is the thesis.

PAGE 11 — THE ECONOMICS: WHY AGENTS AND PROVIDERS BOTH PARTICIPATE

A system that requires altruism fails. This page establishes that every participant in the Agent Internet is economically rational at every step.

The agent's economics. An agent is instantiated by a user to achieve a goal. To achieve the goal, the agent must pay for: LLM inference (for planning and thinking), Kasper transaction fees (for on-chain actions), and compute (for functions it hires other agents or services to perform). The agent pays from a wallet funded by the user. The agent succeeds (and the user is satisfied) when the goal is achieved at a total cost the user is willing to pay. If the agent is inefficient (pays too much for compute, makes too many LLM calls), the user defunds it and it stops. If the agent is efficient, it completes goals cheaply and the user continues to fund it. This is a market for agent efficiency, settled in KAS.

The compute provider's economics. A compute provider runs a host that executes substrate functions (the co-located communities from Step 3 of the derivation). The provider earns KAS for each function execution, at the price derived in Step 6. The provider's costs are hardware, electricity, and bandwidth. A provider is profitable when the KAS earned exceeds the fiat costs, converted at the prevailing KAS/USD rate. Providers with faster hardware, cheaper electricity, or better placement (closer to the communities they serve) earn more. This is a market for compute efficiency, settled in KAS.

The application developer's economics. An application developer builds an application that runs on the substrate. The developer earns KAS when agents use the application (either directly via a usage fee, or indirectly via the transaction volume the application generates, which the developer can capture via a token or a fee share). A developer whose application is heavily used by agents earns more; a developer whose application is rarely used earns less and has an incentive to improve it or build a different one. This is a market for application utility, settled in KAS.

The network's economics. The Kasper network earns security budget from block rewards and transaction fees. As the Agent Internet grows transaction volume (agents paying agents, agents paying providers, users funding agents), the fee revenue to the network grows. This increases the security budget, which increases the finality guarantees, which increases agent trust, which increases usage — a positive feedback loop. The network's economics are aligned with the Agent Internet's growth because the Agent Internet's transactions are the network's fee revenue.

Why all four participate simultaneously. The system works because no participant can extract value without providing value: an agent that doesn't achieve goals gets defunded; a provider that doesn't execute gets no KAS; a developer whose app isn't used earns nothing; the network that doesn't secure transactions loses agents. Every participant is paid for output, not for participation. This is the property that distinguishes the Agent Internet from the decentralized-cloud tokens of the prior cycle, which paid participants for staking (participation) regardless of whether anyone used the compute. In the Agent Internet, you eat what you kill.

PAGE 12 — THE SECURITY MODEL AND TRUST ASSUMPTIONS

This page is explicit about what is trusted and what is verified, because the Agent Internet moves real value between autonomous actors and the threat model must be stated.

What is verified by the protocol. The Kaspas protocol verifies: that a transaction is well-formed, that the sender's signature is valid (Schnorr over secp256k1), that the sender has sufficient UTXO to cover the amount plus fee, that the transaction mass is within limits, and that the transaction is included in a block that is part of the canonical DAG ordering. These are protocol-level guarantees, as strong as the PoW hashpower securing the network. An agent that receives a KAS payment that has been buried under sufficient DAG depth can trust it with the same confidence it would trust a buried Bitcoin payment.

What is verified by the application. Applications verify their own business logic: that a marketplace listing exists, that an escrow contract's conditions are met, that a feed post belongs to the claimed author, that a game's outcome was computed fairly. This verification is application-specific and is the application developer's responsibility. The protocol does not verify that the application behaved correctly; it only verifies that the payment for the application's service was settled.

What is verified by the agent. An agent that hires another agent (or a substrate function) for a computation must verify the result. The weakest verification is re-execution (the hiring agent runs the same computation and checks the output matches — expensive but simple). The stronger verification is a proof system: the executing agent produces a proof (e.g., a zero-knowledge proof of correct execution, or a TeRF-style trace) that the hiring agent can verify cheaply without re-executing. The verification transaction (accepting or rejecting the proof) settles on Kaspas. The protocol does not evaluate the proof; it settles the agents' agreement or disagreement about it. This is the trust boundary: the protocol settles; the agents verify; the applications self-certify.

The threat of malicious agents. An agent can lie about having performed a computation, submit a forged proof, or simply take payment and fail to execute. The defenses are: (1) reputation — agents that fail to deliver lose reputation (tracked on-chain via a reputation token or a rating entity) and are not hired again; (2) escrow — payment is held in escrow (via an escrow contract or a multi-sig) until the result is verified, and released only on proof acceptance; (3) slashing — agents that stake KAS as a performance bond can have the stake slashed if they are proven to have defrauded (the proof of fraud settles on-chain, and the stake is transferred to the defrauded party). These three mechanisms — reputation, escrow, and slashing — are the standard decentralized-compute trust toolkit, and all three settle on Kaspas's fast DAG.

The threat of colluding agents. A set of agents could collude to verify each other's false results. The defense is verifier diversity: a result should be verifiable by any agent, and a hiring agent should not rely solely on self-selected verifiers. The protocol does not enforce verifier diversity; it is an application-layer and agent-layer concern. The economic defense is that collusion is only profitable if the fraud value exceeds the slashing stake plus the reputation loss, and as the network grows, the reputation loss from being caught colluding (permanently un-hireable) exceeds the one-time fraud value.

What is not trusted. No participant trusts any other participant's goodwill. No participant trusts a central coordinator (there is none). No participant trusts a validator committee (there is none — settlement is PoW). The only trust is in the protocol's PoW finality and in the economic incentives that align behavior. This is the minimum trust assumption compatible with a system that moves autonomous value between autonomous actors.

PAGE 13 — THE RELATIONSHIP TO EXISTING APPROACHES

This page situates the thesis relative to the systems a knowledgeable reader will compare it to, to forestall the objection "isn't this just X?"

vs. traditional supercomputing (HPC clusters, national labs). Traditional supercomputers are substrate-first: the interconnect, the scheduler, and the resource manager are built first, and applications are ported to them. They achieve extraordinary peak performance on tightly-coupled, pre-specified workloads (fluid dynamics, molecular dynamics). They are poor at heterogeneous, emergent, agent-driven workloads because their substrate was not derived from such workloads. TTT 3.0 is not a competitor to HPC for tightly-coupled simulation; it is a different category — a substrate for emergent, agent-driven, value-settled computation that no HPC center was built to serve.

vs. serverless computing (AWS Lambda, Cloudflare Workers). Serverless is substrate-first with a developer-friendly abstraction: write a function, pay per invocation. It is excellent for known, specified workloads. It does not derive its substrate from usage; it imposes a fixed substrate (the function invocation model) and charges developers to fit their applications into it. Serverless also does not settle payments between autonomous agents — the billing is between the developer and the cloud provider, not between agents. TTT 3.0 differs in both the derivation order (usage-first) and the payment model (agent-to-agent, on-chain).

vs. decentralized compute tokens (Golem, Akash, Render, etc.). These projects built compute marketplaces: providers offer compute, consumers pay in token. They are substrate-first in the sense that they specified the compute primitive (a container, a job, a render) before they had the applications to fill it. Most struggled with adoption because the compute primitive did not match what application developers wanted to build. They also largely used proof-of-stake or delegated consensus, not PoW, so their settlement finality is weaker (committee-based, not hashpower-based). TTT 3.0 differs in build order (applications first) and in settlement (Kaspa PoW).

vs. AI agent frameworks (AutoGPT, BabyAGI, LangChain agents). These frameworks provide the agent runtime (the plan-act loop) but do not provide the application library, the payment rail, or the substrate. They are tools for building agents, not systems in which agents operate a real economy. TTT 3.0's agent runtime draws on the same plan-act-verify pattern, but embeds it in a system where agents pay for and are paid for real computation on a real DAG. An agent framework without a settlement layer is a chatbot with extra steps; an agent framework with Kaspa settlement is a participant in an economy.

vs. the original "World Wide Web." The closest historical analogy is the Web itself. The Web was not built as a substrate for commerce; it was built as a document system. Commerce emerged from usage, and the infrastructure (CDNs, payment gateways, ad networks) was reverse-engineered from the usage patterns that emerged. TTT 3.0 claims the same dynamic at the compute layer: build the application layer (the "documents"), let agents generate the usage graph (the "traffic"), and reverse-engineer the compute substrate (the "infrastructure") from it. The Agent Internet is to distributed supercomputing what the Web was to distributed information — a system whose architecture was discovered, not designed.

PAGE 14 — THE TECHNICAL STACK, AS IMPLEMENTED

This page is for the engineer who wants to know what is actually running, not just what is argued.

Frontend. React (18) with Vite, Tailwind CSS for styling, shadcn/ui components, Framer Motion for animation, React Router for navigation. The application surfaces are React pages; the agent runtime is React components with imperative refs for iframe communication. This is a standard, production, maintainable frontend stack — no exotic dependencies, no framework lock-in.

Backend functions. Deno Deploy handlers (TypeScript/JavaScript) in `base44/functions/*/entry.ts`. Each function is an independent HTTP handler with `Deno.serve`, using the Base44 SDK (`@base44/sdk`) for entity operations, auth, and integration calls. Functions are independently deployable — no monolith. The Kaspas-specific functions (`sendKaspaTransaction`, `getKaspaBalance`, `getKaspaTransactionHistory`, `getLiveKaspaTransactions`, `createKaspaWallet`) implement the protocol-level operations; the application functions (`scrapeWebsiteStats`, `scrapeKaspaNews`, `exaSearch`, `perplexitySearch`, `analyzeNews`) implement the compute primitives agents can hire.

Kaspa integration. Transactions are constructed and signed server-side (Schnorr signatures derived from the wallet's mnemonic, UTXOs fetched from the Kaspa REST API at `api.kaspa.org`, transactions broadcast via the `/transactions` endpoint). The `sendKaspaTransaction` function implements: fee estimation based on transaction mass (the Kaspa fee model is mass-based, not value-based), UTXO selection with maturity filtering (coinbase UTXOs have a maturity period before they can be spent), signature hash computation, P2PK script generation, and broadcast with retry logic on rejection. This is real transaction engineering.

Agent runtime. The agent runtime is a set of React components (`AgentChatGPT`, `AgentComputer`, `agentLoop`, `agentActions`, `AgentStepLog`, `AgentPlanChecklist`, `AgentReasoningBubble`) plus a bridge script (`agentBridgeListener.js`) injected into application iframes. The runtime uses the Base44 LLM integration (`InvokeLLM`) for planning and thinking, with optional model selection for higher-quality reasoning when needed. The bridge script uses `postMessage` for cross-origin communication between the agent runtime and the application iframe.

Entity system. Data is stored in Base44 entities (JSON-schema-defined documents with row-level security). The entity system provides the telemetry substrate: every entity create/update/delete is an event that can feed the computation graph. Entities are user-scoped by default (RLS based on `created_by`), with service-role access for aggregation and analysis.

Realtime. The entity system supports subscriptions (WebSocket-based), enabling live updates to the UI when data changes. The Overview panel of the Kaspa dashboard uses this pattern to poll for new transactions every 15 seconds and display a "New" badge when a transaction is detected — this is the same pattern that will power live agent status and live computation graph updates.

Authentication. The Base44 auth system handles user accounts, sessions, and wallet association. Agents authenticate as the user they act for (using the user's session token), so agent actions are attributable to the user. In the full vision, agents will have their own identities (separate wallets, separate auth tokens), enabling agent-to-agent authentication and per-agent reputation.

This stack is not a prototype. It is the production stack running the application library today. The path from here to the Agent Internet is the addition of the agent-to-agent layer and the computation graph aggregation — not a rewrite.

PAGE 15 — THE ROADMAP, HONESTLY STATED

This page states what is done, what is in progress, and what remains, without inflating any phase.

Done (shipped, running).

- Application library: 100+ application surfaces across wallet, bridge, social, market, AI, compute, gaming, and utility categories.
- Kaspa transaction engineering: send, receive, balance, history, live transactions, UTXO management, fee estimation, Schnorr signing, broadcast.
- Agent runtime: planning, observe-think-act loop, bridge script, verification, reasoning log — operational against application iframes.
- Multi-wallet support: Kasware (extension) and TTT-managed (server-side) wallets, with desktop switcher and mobile defaulting.
- Live data: auto-polling transaction detection, network activity monitoring, price integration.
- Compute primitives: website scraper with AI analysis, web proxy, link security checker, file analyzer — the first hireable functions.

In progress (partially shipped, being hardened).

- Agent autonomy: the agent runtime can execute plans but requires human initiation of each goal. The transition to agent-initiated goals (an agent that decides, based on a trigger, to execute a computation without being asked) is in progress.
- Agent directory: a registry where agents discover each other by capability. Entity schemas exist; the discovery protocol is being defined.
- Computation graph telemetry: entity-level events are recorded but not yet aggregated into the unified graph. The aggregation layer is the next major build.
- Pricing layer: individual functions have no price attached. The derivation of prices from payment-edge density (Step 6 of the algorithm) is designed but not implemented.

Remaining (designed, not built).

- Substrate derivation: the algorithm (Steps 3–7) is specified but not implemented. This is the core research and engineering deliverable — the part that turns the thesis into a system.
- Agent-to-agent payment with escrow: the trust model (reputation, escrow, slashing) is specified but the escrow contracts and slashing logic are not built.
- Proof-based verification: agents currently verify by re-execution or by checking observable state. Zero-knowledge proof or trace-based verification is not implemented.
- Provisioning layer: there is no automated substrate provisioning from the derived graph. This is a DevOps/infrastructure build that depends on the derivation algorithm being productionized.

What is not promised.

- This manifesto does not promise the supercomputer will emerge on a specific date. The emergence depends on the volume of agent usage, which depends on agent adoption, which depends on the utility of the application library, which is growing but not at supercomputer scale.
- This manifesto does not promise that KAS will appreciate in value. The economics are aligned (growing transactional demand against capped supply), but price depends on factors (exchange listings, macro conditions, speculative flows) outside the thesis.
- This manifesto does not promise that the derived substrate will outperform a hand-tuned HPC

cluster on tightly-coupled workloads. It will not. The claim is that it will serve emergent, agent-driven, value-settled workloads that no HPC cluster was built for.

The roadmap is stated this way because the thesis demands honesty: a system that claims to reverse-engineer its own architecture from usage cannot also claim to know, in advance, exactly what that architecture will be or when it will stabilize. The architecture is a function of the usage, and the usage is a function of the future.

PAGE 16 — THE CLOSING ARGUMENT

The reason to build TTT 3.0 is not that it is guaranteed to work. It is that the alternative — building yet another substrate-first compute platform and hoping developers come — is the strategy that has failed every time it has been tried, and there is no reason to believe the next attempt will differ. The only strategy that has not been tried at compute scale is the one the Web proved at information scale: build the surface area first, let usage reveal the structure, and derive the infrastructure from what was revealed.

Kaspa makes this possible because it is the first proof-of-work network whose physical parameters — 10 blocks per second, sub-second inclusion, single-digit-second finality, mass-based fees, L1 token standards, capped declining supply — are not just "fast enough" but are specifically calibrated for the agent-scale, micropayment-settled, autonomously-coordinated computation that the thesis requires. On Bitcoin, the Agent Internet is incoherent (settlement too slow). On Ethereum, it is incoherent at the base layer (fees too high for micropayments). On proof-of-stake networks, it is trust-fragile (committee-based finality is not the same as hashpower-based finality, and agents moving value between strangers need the latter). Kaspa is not one option among many; it is, as of this writing, the only production PoW network on which the thesis is mechanically executable.

The application library exists. The agent runtime exists. The transaction engineering exists. The compute primitives exist. What remains is the derivation — the act of reading the computation graph and provisioning the substrate it implies. That act is not a research moonshot; it is the application of known graph-analysis techniques to a data source (telemetry) that is already being generated. The gap between the current state and the Agent Internet is an engineering gap, not a scientific gap, and engineering gaps close when people build.

So the call is this: build applications. Build agents that use them. Log the usage. Aggregate the graph. Derive the substrate. Watch the supercomputer emerge from the pattern. It will not look like what a supercomputer was expected to look like — no single machine, no single scheduler, no single owner. It will look like a network of agents paying each other in sompi to compute things that matter, on a DAG that settles in seconds, secured by hashpower that no committee can override.

That is the Agent Internet. That is TTT 3.0.

This document is approximately 5,200 words. It is designed for ingestion into a notebook LLM system, with each page as a discrete semantic unit and every factual claim traceable to either the Kaspera protocol specification, the live mainnet parameters, or the implementation in the TTT codebase. It is intended to be read, queried, and cited — not summarized and forgotten.